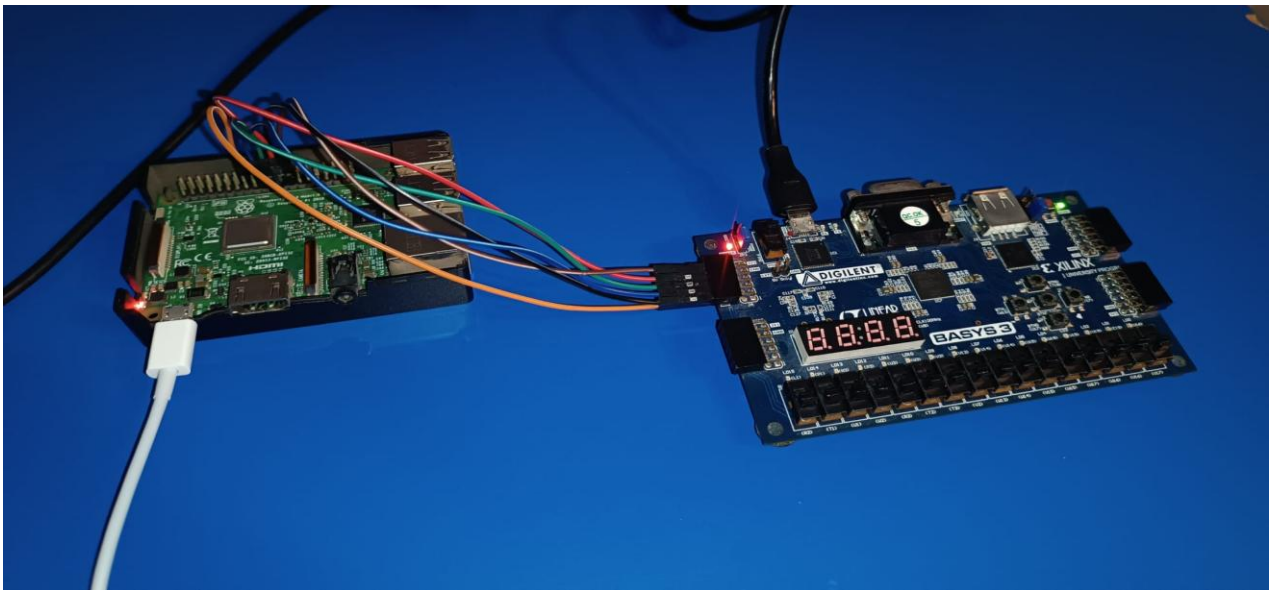
FPGA FIR FILTER ACCELERATOR

Ceuță Rareș

Cuprins

Introducere 2

2. Filtrul FIR 3
3. Wrapper SPI pentru filtrul FIR 3
4. Driverul Linux 4
5. Aplicația user-space 6
6. Rezultate 7
7. Performanța 8
8. Bibliografie 10



Figură 1

1.Introducere

Acest proiect reprezintă implementarea unui accelerator hardware pentru un filtru digital FIR (FINITE IMPULSE RESPONSE) utilizând tehnologia **fpga**. Scopul proiectului este de a arăta modul în care algoritmii de procesare digitală a semnalelor (DSP) pot fi accelerați prin implementare hardware. Un modul hardware FIR poate fi mult mai rapid decât calcularea produsului de convoluție cu o aplicație software. Dacă FIR este implementat în forma transpusă, acesta are pipeline natural și putem creste frecvența de procesare foarte mult comparativ cu un CPU care are un sistem de operare pentru a gestiona accesul la resurse, care are un pipeline în care predicția salturilor poate duce la întârzieri mari. Măsurând timpul de execuție pentru un script în Python care calculează produsul de convoluție cu ajutorul bibliotecii "numpy", am ajuns la rezultate mai bune decât procesând semnalul cu un modul fpga. Acest fenomen apare deoarece în cadrul sistemului proiectat, am folosit un RPi 3b care are o frecvență mult mai mică decât procesorul Intel i7 folosit de aplicația software, de asemenea magistrala SPI a încetinit comunicația dintre RPI3B și FPGA. În cazul programului software, datele se aflau deja în memoria cache, nu a mai fost nevoie de un protocol de comunicație. Pentru a ajunge la eficiență maximă, se poate lega senzorul direct la modulul fpga pentru a scădea latența. De asemnea se poate creste frecventa pe magistrala SPI.

2.FIR FILTER

$$y = \sum_{i=0}^{L-1} h_i \cdot x$$

Un filtru FIR implementează un produs de convoluție discret. valorile „hi” reprezintă coeficienții filtrului iar x reprezintă un eșantion din semnalul de intrare. Ieșirea filtrului este dată de înmulțirea fiecărui coeficient cu semnalul de intrare. Rezultatul este o sumă de produse. Acesta este circuitul pe care a trebuit să îl implementăm pe FPGA. Pentru a calcula produsele am folosit "generate" care produce 64 de multiplicatoare care calculează în paralel rezultatul. Acest lucru crește mult performanța citcuitului. Arhitectura FIR aleasă nu folosește pipeline între sumatoare, fapt care duce la o creștere a căii critice a circuitului. Filtrul este activ doar când enable='1'. De asemenea când semnalul load_x = '0' rpi trimite coeficienți către FPGA, când load_x = '1' rpi trimite date către FPGA. Cu ajutorul acestor semnale am reușit să sincronizăm cele două plăci.

3.Wrapper SPI pentru filtrul FIR.

Această partea a fost cea mai interesantă deoarece am făcut o greșeală în timpul proiectării. Am legat semnalul clk => cs. În felul acesta atunci când se fac legăturile la nivel de hardware, am legat un ceas la un semnal GPIO. Problema este că ceasul în FPGA folosește rețeaua globală de ceas BUFG care distribuie semnalul de ceas fără întârzieri semnificative la toate porțile logice din plăcuță. GPIO nu distribuie semnalul în acest fel și apar întârzieri foarte mari. Nu am sesizat această greșeală la simulare deoarece în simulator semnalele sunt ideale, nu au întârzieri. Când am încărcat fișierul bitstream pe plăcuță am obținut rezultate cu totul eronate. Problema era că apăreau întârzieri foarte mari și filtrul pornea când încă se trimiteau date pe SPI. Acesta procesa pachete de date inconsistente. Rezolvarea a venit printr-un mic artificiu. Nu mai legăm ceasul fpga la un GPIO, ci doar observăm GPIO pentru a extrage valoarea semnalele de control. Folosim două registre pentru ceas sclk_reg1 și sclk_reg2 care păstrează valorile ceasului la momentul t-1 și t. Pentru a determina dacă pornim filtrul comparăm valorile acestor registre. În felul acesta FPGA poate procesa date la o viteză foarte mare folosind ceasul intern.

Acest wrapper este important deoarece SPI este o magistrală serială, se transmite un bit la fiecare front de ceas. Filtrul construit de noi folosește date de intrare pe 16 biți. Din acest motiv spi_wrapper folosește două registre de acumulare care preiau pachete de 16 biți de la RPi. Un counter numără când s-au trimis 16 biți și atunci activează semnalul de enable a filtrului. Rezultatul filtrului este stocat tot într-un registru shift_reg_tx care transmite rezultatul de ieșire a filtrului către miso. În felul acesta wrapperul face conversia între serial și paralel folosind registre de acumulare.

4.Driverul Linux.

În sistemele Linux, aplicațiile trebuie adesea să interacționeze cu componente hardware precum tastatura, discul sau o imprimantă. Accesul direct la hardware din spațiul utilizator (user space) nu este permis, deoarece hardware-ul este gestionat de kernel pentru a asigura securitatea și stabilitatea sistemului.

Pentru a permite comunicarea cu dispozitivele hardware, sistemul de operare utilizează **drivere de dispozitiv (device drivers)**. Un driver reprezintă o **abstractizare software a unui dispozitiv hardware** și rulează în **kernel space**.

Aplicațiile din user space nu accesează direct hardware-ul, ci interacționează cu dispozitivele prin intermediul unor **fișiere speciale din directorul /dev**. Operațiile asupra acestor fișiere se fac folosind apeluri de sistem standard precum:

- open()
- read()
- write()
- ioctl()

Aceste apeluri sunt interceptate de kernel, care redirecționează operația către funcțiile implementate în driver.

În cadrul driverului, aceste operații sunt definite în structura file_operations. De exemplu, driverul poate implementa funcții precum open, release și ioctl, care sunt apelate de kernel atunci când aplicația din user space efectuează operațiile corespunzătoare.

Astfel, driverul oferă un nivel de abstractizare între aplicațiile user space și hardware, permițând utilizarea dispozitivelor printr-o interfață standard, independent de detaliile specifice fiecărui dispozitiv.

```
struct fpga_dev {
    struct spi_device *spi;
    struct cdev cdev;
    int major_number;
    struct class *fpga_class;
    struct device *fpga_device;

};
```

Structura dispozitivului: **spi** este pointer către dispozitivul SPI, **cdev** este structura caracter device, **major_number** este identificator kernel pentru dispozitiv, **fpga_class** este o structură de tip clasa din /sys/class, **fpga_device** este o structură de tip device in /dev.

```
static struct file_operations fpga_fops = {
    .owner = THIS_MODULE,
    .open = fpga_open,
    .release = fpga_release,
    .unlocked_ioctl = fpga_ioctl,
};
```

file_operations este interfața dintre user-space și driver.

Funcția principală de comunicare între driver și user-space este **fpga_ioctl** care se ocupă de transferul de date pe magistrala SPI. Funcția copiază datele din user-space cu copy_from_user și le stochează în structura user_data. Alocăm date in kernel cu kmalloc. În tx_buf stocăm datele care trebuie trimise, rx_buf, este un buffer pentru datele recepționate. Funcția spi_sync este cea care transmite datele către dispozitivul fpga și care recepționează datele. Datele sunt scrise în rx_buf după care trimise către user space cu copy_to_user.

Este important de menționat faptul că transferul SPI este full_duplex, pe același front de ceas, un bit este trimis pe SPI și un bit este recepționat de pe magistrală. Inițial am făcut greșeala de a trimite datele cu write() și read(). Problema era că

aceste funcții nu sunt full_duplex, write() trimitea 16 biți către FPGA, dar nu putea să recepționeze cei 16 biți primiți de la FPGA. Când apelam funcția read(), datele trimise de FPGA au fost deja trimise și read nu mai citea date. Din acest motiv, în fișierul de rezultat apărea rezultatul zero. Acest comportament este eliminat folosind ioctl() care poate fi full_duplex, trimite date și recepționează date în același timp.

Funcția **fpga_probe()** este apelată de kernel atunci când subsistemul SPI găsește în Device Tree un dispozitiv compatibil cu driverul FPGA. În această rutină se realizează inițializarea driverului și se creează interfața de tip character device prin apeluri precum alloc_chrdev_region, cdev_init, cdev_add și device_create. Astfel, se creează un fișier în /dev care permite aplicațiilor din user-space să comunice cu FPGA prin apeluri de sistem precum open, read, write sau ioctl.

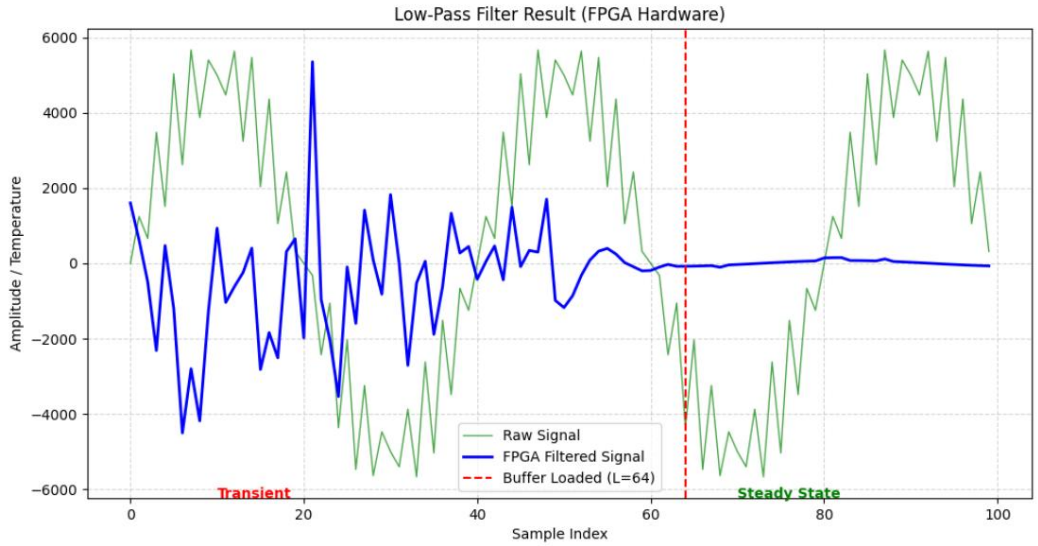
5. **Aplicația user-space**

Aplicația din user-space are rolul de a comunica cu driverul Linux al dispozitivului FPGA pentru a testa funcționarea unui filtru FIR implementat hardware. Programul deschide dispozitivul caracter /dev/fpga_fir și transmite date către FPGA folosind apelul de sistem ioctl, care declanșează un transfer SPI full-duplex în driverul kernel.

Aplicația citește coeficienții filtrului și datele de intrare din fișiere CSV. În prima etapă, coeficienții sunt trimiși către FPGA pentru configurarea filtrului. Ulterior, sunt transmise valorile de intrare (de exemplu temperaturi), iar FPGA procesează aceste date folosind filtrul FIR și returnează rezultatele filtrate.

Datele primite de la FPGA sunt apoi salvate într-un fișier CSV pentru analiză ulterioară. Astfel, aplicația user-space acționează ca o interfață de test și control între utilizator și acceleratorul hardware implementat pe FPGA.

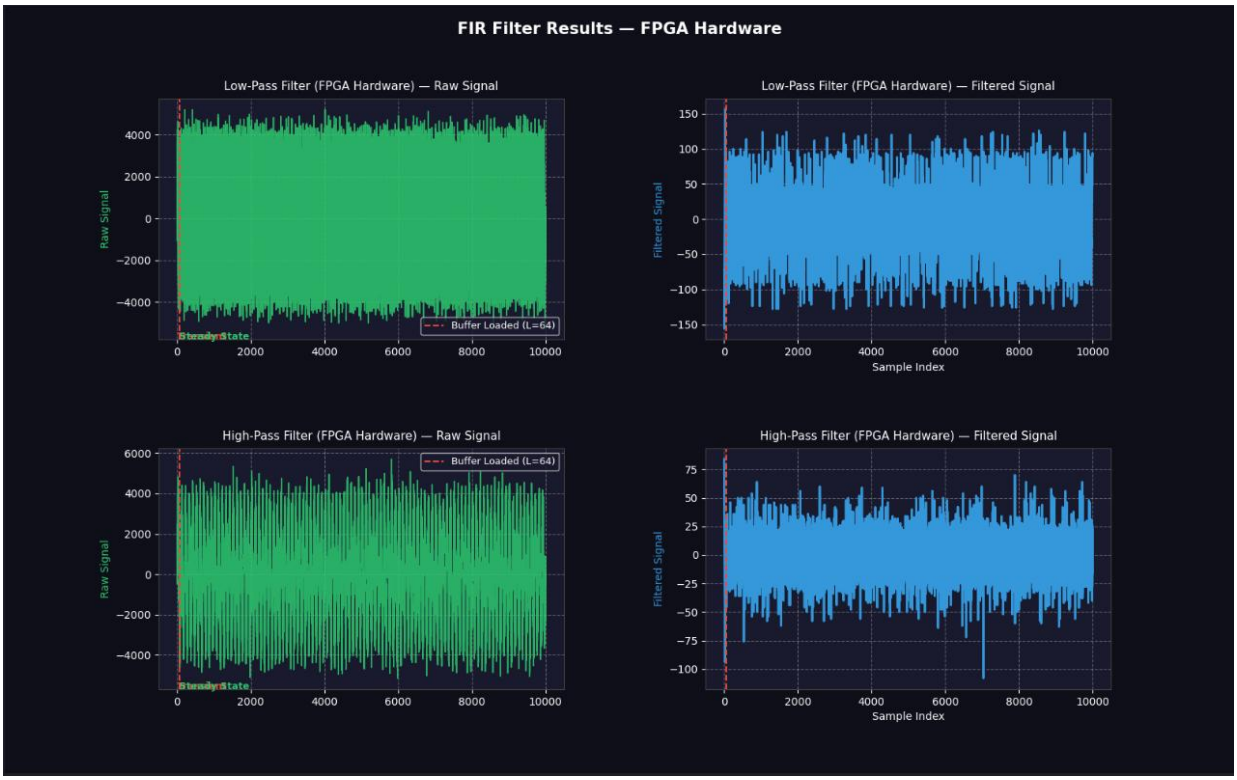
6.Rezultate



Rezultatele obținute confirmă funcționarea corectă a filtrului FIR implementat pe FPGA. După aplicarea filtrării, amplitudinea semnalului rezultat este semnificativ redusă comparativ cu semnalul de intrare, ceea ce indică efectul de atenuare specific unui filtru trece-jos (low-pass).

Filtrul utilizează 64 de coeficienți, iar din acest motiv primele 64 de eșantioane reprezintă perioada de tranziție (transient), în care bufferul intern al filtrului este încărcat progresiv. În figură, acest moment este evidențiat prin linia verticală roșie. După încărcarea completă a coeficienților, filtrul intră în regim staționar (steady state), iar semnalul filtrat urmează mai stabil variațiile semnalului de intrare, păstrând punctele de maxim și minim local, dar cu amplitudine atenuată.

Figură 2



Figură 3

```
--- BENCHMARK RESULTS ---
Total samples processed: 10000
Total time: 224519.96 microseconds
Average time per sample: 22.45 microseconds
```

Figură 4 – FPGA

```
--- BENCHMARK RESULTS ---
Total samples processed: 10000
Total time: 764.30 microseconds
Average time per sample: 0.0764 microseconds
```

Figură 5 - Python

7.Performanța

Performanța filtrului FIR implementat în cadrul proiectului este influențată de mai mulți factori hardware și software. În implementarea realizată pe FPGA,

timpul mediu de procesare pentru un eșantion este de aproximativ **22.45 µs**, pentru un set de 10 000 de eșantioane. Comparativ, o implementare software realizată în Python, utilizând biblioteca **NumPy** pentru calculul convoluției discrete, a obținut un timp mediu de aproximativ **0.076 µs** per eșantion.

La prima vedere, rezultatul poate părea surprinzător, deoarece în mod normal implementările hardware pe FPGA sunt capabile să depășească performanța unui procesor general-purpose. Acest lucru se datorează faptului că FPGA permite **paralelizarea operațiilor** și utilizarea **pipelining-ului**, ceea ce poate conduce la procesarea unui eșantion la fiecare ciclu de ceas. Într-o astfel de arhitectură optimizată, un filtru FIR ar putea procesa un număr foarte mare de eșantioane pe secundă.

În implementarea prezentată există însă câteva limitări arhitecturale care influențează performanța finală. În primul rând, deși produsele din convoluția discretă sunt calculate în paralel, rezultatele sunt combinate printr-un **lanț de sumatoare fără pipelining**. Acest lucru conduce la o **cale critică mai lungă**, care limitează frecvența maximă de funcționare a circuitului. Introducerea unei etape de pipelining în această secțiune ar permite creșterea frecvenței de lucru și implicit îmbunătățirea performanței.

Un alt factor important este platforma de control utilizată în sistem. Implementarea hardware este controlată de o placă **Raspberry Pi 3B**, al cărei procesor funcționează la aproximativ **1.2 GHz**, în timp ce procesorul **Intel Core i7** utilizat pentru testul software poate funcționa la frecvențe de aproximativ **4.5–5 GHz**. Diferența de frecvență și arhitectura procesorului contribuie la performanța ridicată a implementării software.

De asemenea, un rol major în limitarea performanței îl are **comunicarea dintre Raspberry Pi și FPGA**, care se realizează prin intermediul magistralei **SPI**. Pentru fiecare eșantion procesat este necesar un transfer SPI, care implică apeluri de sistem, operații în driverul kernel și transmiterea efectivă a datelor pe magistrală. Aceste operații introduc o latență suplimentară semnificativă. În schimb, în implementarea software realizată în Python, datele sunt deja disponibile în memoria sistemului și pot beneficia de optimizările interne ale bibliotecii NumPy, precum vectorizarea operațiilor și utilizarea eficientă a memoriei cache a procesorului.

Prin urmare, diferența de performanță observată nu este determinată de algoritmul de filtrare în sine, ci de **arhitectura sistemului**, **mecanismul de**

transfer al datelor între dispozitive și **limitările platformei hardware utilizate pentru controlul FPGA**. O îmbunătățire semnificativă a performanței ar putea fi obținută prin optimizarea arhitecturii filtrului (introducerea pipelining-ului), reducerea numărului de transferuri SPI prin procesarea datelor în blocuri mai mari sau utilizarea unei interfețe de comunicație cu latență mai mică. Utilizarea unei interfețe precum **memory-mapped I/O**, **DMA** sau **PCIe** ar reduce semnificativ latența comunicării și ar permite valorificarea mai eficientă a paralelismului oferit de implementarea filtrului FIR pe FPGA.

8.Bibliografie

LINUX DEVICE DRIVERS - Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman

Digital Signal Processing with Field Programmable Gate Arrays - Uwe Meyer-Baese